Detalhamento do Conteúdo Obrigatório

BootCamp: Entrega Inicial — Etapa 1

Aluno: Marcelo Augusto de Abreu Santos

Disciplina: Bootcamp II

Projeto: Organizador de Estudos (CLI)

Repositório público no GitHub: https://github.com/MarcelinhoJC/organizador-estudos

1. Visão geral do projeto

O projeto "Organizador de Estudos" é uma aplicação de linha de comando (CLI), desenvolvida em Java 17 com Maven, que ajuda estudantes a organizar suas tarefas de estudo. A aplicação permite cadastrar, listar, concluir, remover e buscar tarefas, cada uma associada a uma matéria, uma prioridade (BAIXA, MEDIA ou ALTA) e um prazo opcional. Os dados são persistidos localmente em um arquivo de texto (tasks.csv), sem necessidade de internet, banco de dados externo ou cadastro de usuário.

2. Problema real e relevância social

Muitos estudantes têm dificuldade em manter uma rotina de estudos organizada, perdendo prazos de provas, trabalhos e revisões por não terem um local simples e centralizado para anotar o que precisa ser feito. Planilhas são trabalhosas de manter e aplicativos completos exigem cadastro, internet e muitas vezes cobrança. O Organizador de Estudos resolve essa dor oferecendo uma ferramenta leve, rápida e totalmente local, executável a partir do terminal, sem barreiras de acesso.

3. Detalhamento de cada requisito obrigatório

A tabela abaixo relaciona cada requisito exigido pelo enunciado da atividade com a forma como ele foi implementado e evidenciado no repositório do projeto.

Requisito	Como foi atendido no projeto
Linguagem e interface	Java 17, aplicação CLI (linha de comando) interativa via menu numerado, implementada em Cli.java e Main.java.
Problema real	Estudantes com dificuldade de manter rotina de estudos não têm um local simples e local (sem internet, sem cadastro) para organizar tarefas por matéria, prioridade e prazo.
Repositório público	Publicado em GitHub, acessível para leitura e correção por qualquer avaliador.
README.md	Contém descrição do problema, solução, público-alvo, funcionalidades, tecnologias, instruções de instalação/execução/testes/lint, versão e autor.
Versionamento semântico	Versão 1.0.0 declarada no pom.xml (tag <version>), seguindo o padrão MAJOR.MINOR.PATCH e documentada no CHANGELOG.md.
Dependências declaradas	Arquivo pom.xml (Maven) declara explicitamente a dependência de testes JUnit Jupiter 5.10.2 e os plugins de build/lint.

Testes automatizados	11 testes JUnit 5 em TaskManagerTest.java, cobrindo caminho feliz, entradas inválidas e casos limite (adicionar, concluir, remover, buscar, persistir/carregar).
Linting / análise estática	Maven Checkstyle Plugin configurado via checkstyle.xml, verificando estilo, imports não usados, blocos vazios, etc.
CI com GitHub Actions	Workflow .github/workflows/ci.yml executa a cada push/pull request: setup do JDK, instalação de dependências, checkstyle:check e mvn test.
PDF final	Este documento (convertido em PDF) e o PDF de entrega contêm nome do aluno, nome do projeto, resumo e link do repositório público.

4. Estrutura do repositório

O projeto segue uma estrutura de pastas organizada, separando código-fonte, testes e configuração:

```
organizador-estudos/
├── src/main/java/org/estudos/
│   ├── Main.java          # ponto de entrada da aplicação
│   ├── Cli.java           # interface de linha de comando (menu)
│   ├── TaskManager.java   # regra de negócio: cadastro, busca, persistência
│   ├── Task.java          # entidade Tarefa
│   └── Priority.java      # enum de prioridade (BAIXA, MEDIA, ALTA)
├── src/test/java/org/estudos/
│   └── TaskManagerTest.java
├── .github/workflows/ci.yml   # pipeline de integração contínua
├── checkstyle.xml             # regras de linting
├── pom.xml                    # versão, dependências e plugins
├── CHANGELOG.md
├── LICENSE
├── .gitignore
└── README.md
```

5. Fluxo de desenvolvimento e qualidade

- **Versionamento com Git:** todo o histórico de desenvolvimento é rastreado via commits, com mensagens descritivas.
- **Qualidade automatizada:** a cada push ou pull request para a branch main, o GitHub Actions executa automaticamente o linting (Checkstyle) e os testes automatizados (JUnit 5), impedindo que código com problemas de estilo ou testes quebrados passe despercebido.
- **Separação de responsabilidades:** a lógica de negócio (TaskManager) é totalmente independente da camada de interface (Cli), o que permite testá-la de forma isolada, sem depender de entrada de console.
- **Persistência simples e transparente:** as tarefas são salvas em um arquivo de texto legível (tasks.csv), facilitando inspeção e depuração.

6. Como executar e validar o projeto

- **Instalar dependências:** mvn install -DskipTests
- **Executar a aplicação:** mvn compile exec:java -Dexec.mainClass="org.estudos.Main"
- **Rodar os testes automatizados:** mvn test
- **Rodar o linting:** mvn checkstyle:check
- **Empacotar em um .jar executável:** mvn package

7. Conclusão

O projeto Organizador de Estudos demonstra, em escala reduzida, um ciclo profissional de desenvolvimento de software: definição de um problema real, implementação funcional, testes automatizados, análise estática de código, documentação clara e integração contínua. Todos os requisitos obrigatórios da atividade foram atendidos e estão evidenciados no repositório público listado acima.